

Computing the Pathwidth and Bandwidth of Solid, Convex Grids *

JOHN ELLIS

*Department of Computer Science
University of Victoria, P.O. Box 3055
Victoria, British Columbia, V8W 3P6, Canada*

September 4, 2023

Abstract

We describe a $O(n^2)$ time algorithm for computing the pathwidth of those subgraphs of two dimensional grid graphs whose periphery is convex and which contain no holes. This extends the set of restricted graphs for which polynomial time algorithms are known for computing pathwidth, which problem is intractable in general. The algorithm uses a simple “sweep line” process, such as is often used in computational geometry. We suggest that this algorithm might permit the computation of approximations to this parameter on more complex graphs by first embedding them in these kinds of grid.

We observe that for a well defined proper subset of these graphs the pathwidth is identical to the bandwidth and that for a larger proper subset $\text{pathwidth} \leq \text{bandwidth} \leq \text{pathwidth} + 1$.

*This work was supported by the Natural Sciences and Engineering Research Council of Canada

1 Introduction

1.1 Background

Pathwidth is a graph parameter whose importance in determining the algorithmic complexity of width bounded instances of intractable problems is well known. The pathwidth of a graph is identical to its vertex separation [15] and the node search number is equal to vertex separation + 1 [16]. Hence these concepts are equivalent. There is a substantial literature surrounding them. Surveys can be found in [2], [7] and [1]. An extensive, annotated bibliography can be found in [11].

The related decision problem is NP-complete and in [18] it is shown that the problem remains NP-complete even for planar graphs of degree 3. Polynomial time algorithms have been given for some simple graph families. For example, [10] showed that the vertex separation of trees can be computed in linear time and gave an $O(n \log n)$ algorithm for computing an optimal layout. In [20, 21] it is shown that optimal layouts for trees can also be computed in linear time. In [8] the methods used in [10] are extended to include linear time computation of optimal layouts for trees and gives an $O(n \log n)$ algorithm for unicyclic graphs. Lower bounds on the pathwidth of some grid-like graphs, including 2-dimensional grids, cylinders and tori, are given in [9]. Since matching upper bounds which are simple functions of the graph dimensions are easily obtained, it follows that the pathwidth of these kinds of graph is easily computed. An $O(n^2)$ algorithm for computing pathwidth of circular arc graphs is given in [22]. The pathwidth of hypercubes in terms of its dimensions is given in [6] and a closed formula for the pathwidth of 3-dimensional grids and for multi-dimensional grids that have relatively large maximum factors are given in [19].

One can deduce from the results in [5] that there is a polynomial time algorithm for outerplanar graphs, because they have fixed treewidth. However, applying that general method seems to yield an algorithm of time complexity $\Omega(n^{11})$. Approximation algorithms for outerplanar graphs are given in [12] and in [3]. In [23], edge search number is considered for *pseudo 3-sided*, *solid*, *ortho-convex grids*, which are a proper subset of the grid family discussed in this paper. Edge search number is not the same as node search number and so is not identical to pathwidth.

In this paper we show that pathwidth of *solid*, *convex grids* can be computed in $O(n^2)$ time by a simple algorithm that uses a *sweep line* process, such as is often used in computational geometry. See for example [13]. Since the embeddings of more complex graphs into grids has been studied, see for example [24], there is the possibility of computing approximations to the pathwidth of more complex graphs by way of embedding them in solid, convex grids, whose pathwidth is then computed via the algorithm given here.

Our analysis is worked in terms of *node search number*, denoted $sn(G)$, rather than directly in terms of pathwidth. We then use the equations $pw(G) = sn(G) - 1$, implied by the equations given above, to obtain the pathwidth.

The *bandwidth* of a graph, denoted $bw(G)$, is, like vertex separation, a linear layout property. In general the only relationship between the pathwidth and the bandwidth of a

graph is $bw(G) \geq pw(G)$. However, it follows from the analysis of the pathwidth problem, presented here, that, for a well defined subset of these solid, convex grids, $pw(G) = bw(G)$ and that, for a larger subset $pw(G) \leq bw(G) \leq pw(G) + 1$.

1.2 Node Search Number

Informally, the search number of a simple, undirected graph is the minimum number of guards necessary to guarantee the capture of a fugitive who can move with arbitrary speed about the edges of the graph. Several versions of the game have been proposed. We are concerned here only with the *node search number*, which is defined as follows.

A *search step* is either the placement of a guard on or removal of a guard from a vertex. Initially, all the edges of the graph are *dirty*. A dirty edge $\{u, v\}$ is *cleaned* when guards are simultaneously occupying both u and v . A clean edge will become *recontaminated* only if the removal of a guard results in a path without guards from a dirty edge to the clean edge. A *search* S for a graph is a sequence of search steps that results in all edges being simultaneously clean with no guards remaining on G . The cost of a search, $cost(S)$, is the maximum number of guards on G over all steps in S . The node search number of a graph $sn(G) = \min\{cost(S) \mid S \text{ is a search for } G\}$. Any search which achieves minimum cost we will refer to as *optimal*. In the rest of the paper we use “search number” to mean “node search number”. When we say “at step t ” we will mean “at the completion of the t^{th} step”.

A most useful result [17], [16], assures us that we do not need to consider searches that permit recontamination. No saving in guards can be gained in that way. We deduce that we need only consider strategies in which no guard on a node incident with a dirty edge is ever removed. This is because if the guard is also incident with a clean edge, then it cannot be removed without causing recontamination but if the guard is incident with nothing but dirty edges and is removed, the original placement was redundant. A search of no greater cost can be created by removing this pair of placement, removal steps. Throughout the rest of the paper we assume all searches are of this restricted form.

A vertex incident with both dirty and clean edges will be called a *critical* vertex and if it is incident with exactly one dirty edge it will be called *uni-critical*. Any critical vertex must be guarded, to prevent recontamination, and there must be a further guard placement before a guard can be removed from a critical vertex. Hence, if at some step in a search there are n critical vertices, $cost(S) \geq n + 1$. We will refer to this observation as the *criticality principle*.

We will also need a process we call the *reversal* of a search S , denoted S^R . See for example [8, Definition 3.1]. S^R is created by reversing the order of the moves in S and replacing every guard placement by a guard removal from the same vertex, and every removal by a placement. As shown in, for example, [8, Lemma 3.1], if S is a search then so is S^R and $cost(S^R) = cost(S)$.

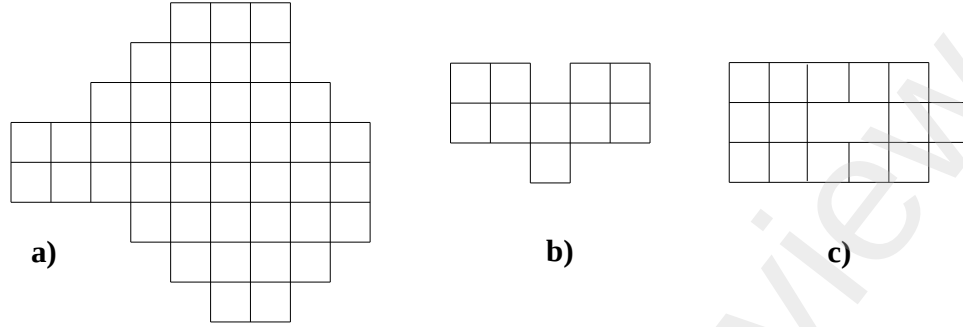


Figure 1: A solid, convex grid and others not so.

2 Terminology and Notation

2.1 Solid, convex grids

We define an (h, w) -grid to be the graph comprising the node set $\{\{x, y\} \mid 1 \leq x \leq h, 1 \leq y \leq w\}$ and the edge set $\{\{(u, v), (x, y)\} \mid |u - x| + |v - y| = 1\}$. Equivalently, the (h, w) -grid is the product of P_h and P_w where P_n is the simple path with n vertices. We work with the standard rectilinear embedding of such a grid in the plane. The edges in the outer face of the embedding we call the *periphery*. Vertices incident with peripheral edges we call *peripheral* vertices. All others are *internal* vertices.

We are concerned only with *solid, convex grids*. These we define to be the biconnected subgraphs of the (h, w) -grids in which every internal vertex is of degree four and whose periphery is convex. Convexity is a property of a clockwise traversal of the perimeter of the grid, while noting the sequence of the left (L) and right (R) turns making up the traversal. The number of right turns equals the number of left turns plus four. The grid is convex if and only if the sequence has the form:

$$R (LR)^* R (LR)^* R (LR)^* R (LR)^* ,$$

where $(LR)^*$ denotes a sequence of zero or more LR's.

For example, the grid in Figure 1 a) is convex and solid, b) is solid but not convex and c) is convex but not solid. Throughout we use the short-hand “grid” to denote “solid, convex grid”. We note that grids are symmetrical with respect to 90° rotations in the plane and 180° rotations in the third dimension about a diagonal. When we say “without loss of generality” we mean we choose one of several configurations that are all equivalent by way of this symmetry.

Figure 2 illustrates our notation for solid, convex grids. $R1$ is the uppermost vertex incident with a leftmost, vertical peripheral edge. $R2$, $R3$ and $R4$ are defined analogously. We call these vertices the *corners* of the graph. The set of peripheral vertices and edges between and including $R1$ and $R2$, we call a *side* and name $S1$. The other sides, $S2$, $S3$ and $S4$, are defined analogously. It follows that all peripheral vertices, except for the corners,

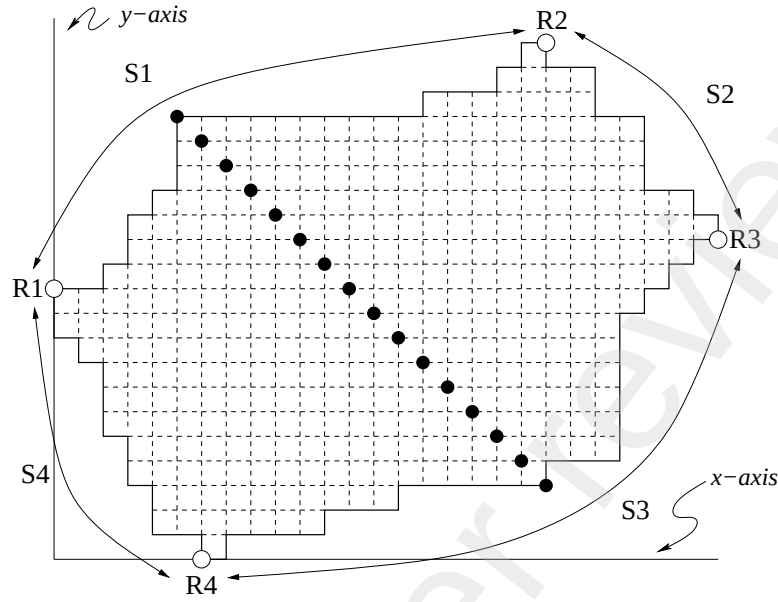


Figure 2: The sides, corners and a diagonal of a solid, convex grid.

are in exactly one side. The corners are in exactly two sides, which sides we call *adjacent* sides. Two sides that are not adjacent we call *opposing* sides.

We use the horizontal line through $R4$ as an x -axis and the vertical line through $R1$ as a y -axis. Then each vertex has (x, y) coordinates relative to an origin at the intersection of these axes. For a vertex u we use the notation u_x and u_y to refer to its x and y coordinates respectively. If $u_x < v_x$ we say that u is strictly to the left of v and v is strictly to the right of u . If $u_y < v_y$ we say that u is strictly below v and v is strictly above u .

It follows immediately from the convexity of the grid and the definition of the four sides, that:

Observation 2.1

$u \in S1$ and $v \in S2$ implies $u_x < v_x$ unless $u = v = R2$,

$u \in S2$ and $v \in S3$ implies $u_y > v_y$ unless $u = v = R3$,

$u \in S3$ and $v \in S4$ implies $u_x > v_x$ unless $u = v = R4$,

$u \in S4$ and $v \in S1$ implies $u_y < v_y$ unless $u = v = R1$,

2.2 Diagonals

A *diagonal* is a set of vertices defined by a maximal line segment in the plane containing diagonally opposing corners of internal faces, whose end points are in the periphery, e.g., the black nodes in Figure 2. Any such set is a separator for the graph. There are two possible diagonal orientations. If the line runs from top left to bottom right we say it is *negatively*

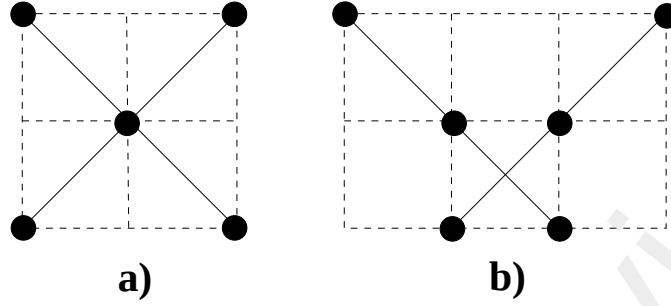


Figure 3: Intersecting Diagonals

orientated. Otherwise we say it is *positively orientated*. We denote the set of negatively orientated diagonals by $\mathcal{D}^-$ and the set of positively orientated diagonals by $\mathcal{D}^+$.

We define the *length* of a diagonal to be the cardinality of the set of vertices in the diagonal and denote it by $|D|$ for any diagonal D . We let D_m^+ denote a diagonal in $\mathcal{D}^+$ such that $\forall D \in \mathcal{D}^+, |D_m^+| \geq |D|$, and we let D_m^- denote a diagonal in $\mathcal{D}^-$ such that $\forall D \in \mathcal{D}^-, |D_m^-| \geq |D|$. Without loss of generality we will assume throughout that $|D_m^-| \geq |D_m^+|$.

Observation 2.2 *The endpoints of a maximum diagonal are in opposite sides, otherwise there exists a longer diagonal whose endpoints are in opposite sides.*

If every pair of vertices u and v that are adjacent in a diagonal are such that $|u_x - v_x| = 1$ we will say that the diagonal is *continuous*, else it is *discontinuous*. Suppose D and D' are in $\mathcal{D}^-$ and defined by lines L and L' respectively. We say that D' is *above* D if, for all points $p \in L$ and $q \in L'$, $p_x = q_x \Rightarrow p_y < q_y$. Similarly, we will say that one diagonal is *below* another and analogously for diagonals in $\mathcal{D}^+$. In Figure 3, the dashed lines represent edges and the solid lines denote diagonals. We will say that two diagonals A and B *intersect* if $A \cap B \neq \emptyset$, Figure 3 a), or $\exists u \in A$ and $\exists v \in B$ such that $\{u, v\}$ is an edge in G , Figure 3 b).

2.3 Exposure, Criticality, Clean and Dirty Edges and Faces

At any step in a search, the set of clean edges define one or more *components*, i.e., maximal connected subgraphs, of the grid. These components are not necessarily biconnected. A face will be said to be a *clean* face if all the edges in that face are clean, else it is a *dirty* face. A clean edge will be said to be *exposed* if it is in a dirty face. Note that a clean edge in the periphery of the graph is not necessarily an exposed edge.

Figure 4 illustrates these concepts. It shows one clean component at some step in a search, in which there are no peripheral graph edges. Both dashed and solid lines represent clean edges. The solid edges are exposed, the dashed edges not so. The shaded faces are clean, the others are dirty. The black vertices are critical, all other not so. Other edges in the grid are not shown. An *exposed path* is a path in which every edge is exposed, e.g., the set of edges in the periphery of a clean component. With each exposed edge we can associate

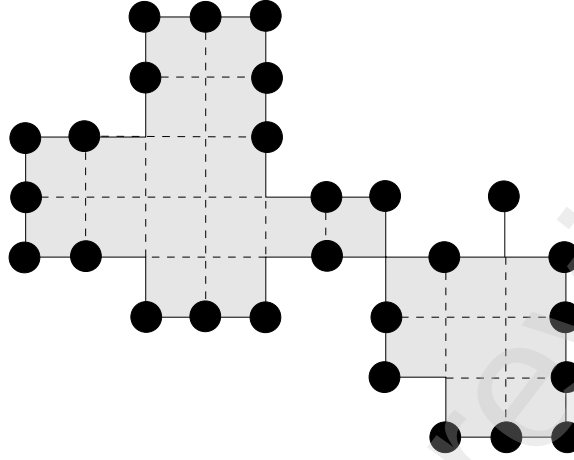


Figure 4: Criticality, clean and dirty edges and faces

a directed edge such that an associated dirty face is on the left. A sequence of such edges forms a *clockwise, exposed path*, e.g., the set of peripheral edges in Figure 4. We use Pr,s to denote a clockwise, exposed path from r to s .

3 Upper Bounds

3.1 The Sweep Line Process

Let G be a solid, convex grid. Consider a *sweep line* process that sweeps a diagonal line of either orientation through the embedding of G in the plane. If the orientation of the line is positive, we call it a positive sweep; if negative then it is a negative sweep. Suppose the line advances in discrete steps, numbered 1 through n . At each step it defines a diagonal, which we call a *sweep set*. Let the *sweep width at step i* be the cardinality of the sweep set at step i . The *negative simple sweep width*, denoted $ssw^-(G)$, of G is $\max_{1 \leq i \leq n} \{\text{negative sweep width at step } i\}$ and analogously for the *positive sweep width*, denoted $ssw^+(G)$. Then the *simple sweep width* of G , denoted $ssw(G)$, is $\min\{ssw^-(G), ssw^+(G)\}$. It follows from the definitions that $ssw^-(G) = |D_m^-|$ and $ssw^+(G) = |D_m^+|$.

Figure 5 shows both a negative and a positive sweep on a solid, convex grid. The positive sweep, Figure 5 a), defines sweep sets containing 3, 6, 8, 7 and 1 vertices at steps 1, 3, 6, 10 and 11 respectively. The negative sweep, Figure 5 b), defines sweep sets with 1, 4, 4, 5, 5, 3 and 1 vertices at steps 1, 3, 6, 9, 13, 15 and 17 respectively. Since 5 and 8 are the maxima over all steps of the positive and negative sweeps, the simple sweep width of that grid is 5.

It can be seen that a search that moves guards through the grid in exactly the same way as the sweep line is a search that requires no more than sweep width + 1 guards. We prove this in Lemma 3.1. We will show that there are instances, such as those shown in Figure 5,

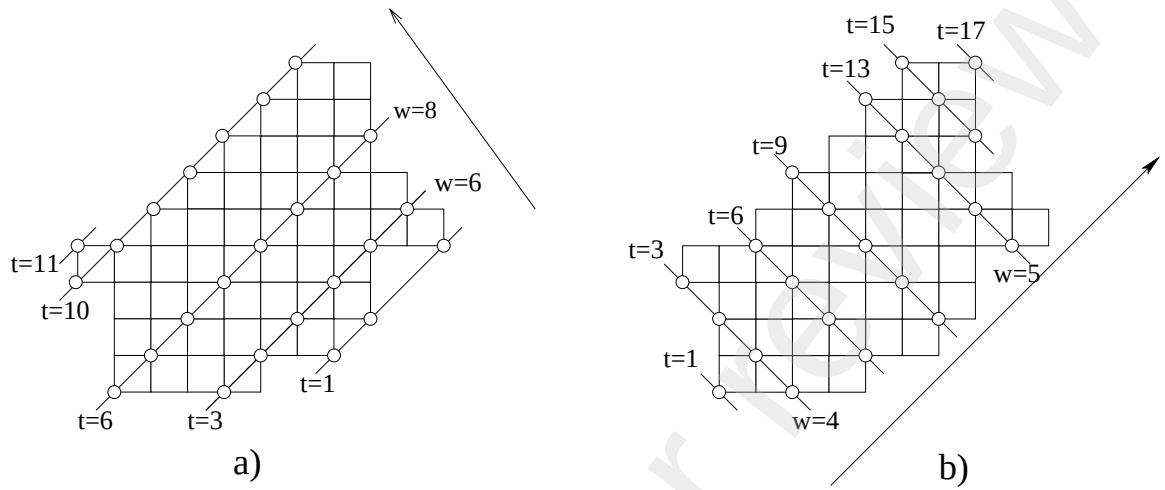


Figure 5: The two sweep orientations

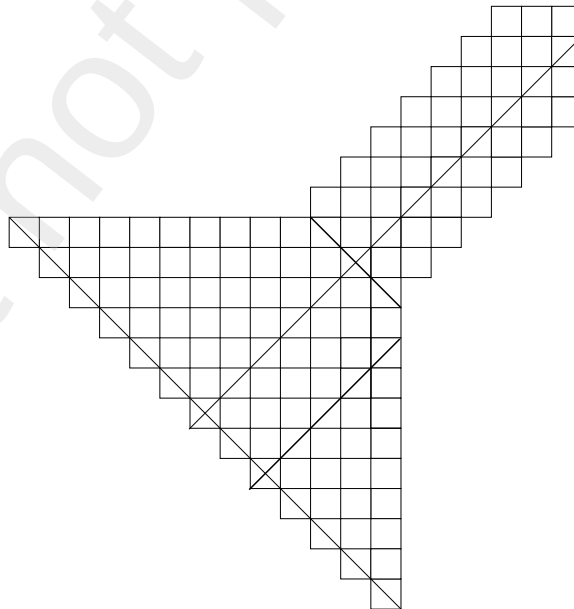


Figure 6: A Composite Search

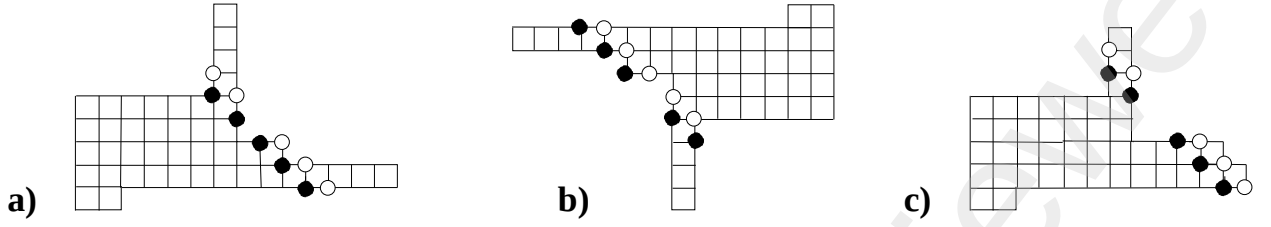


Figure 7: Maximum diagonals can be discontinuous

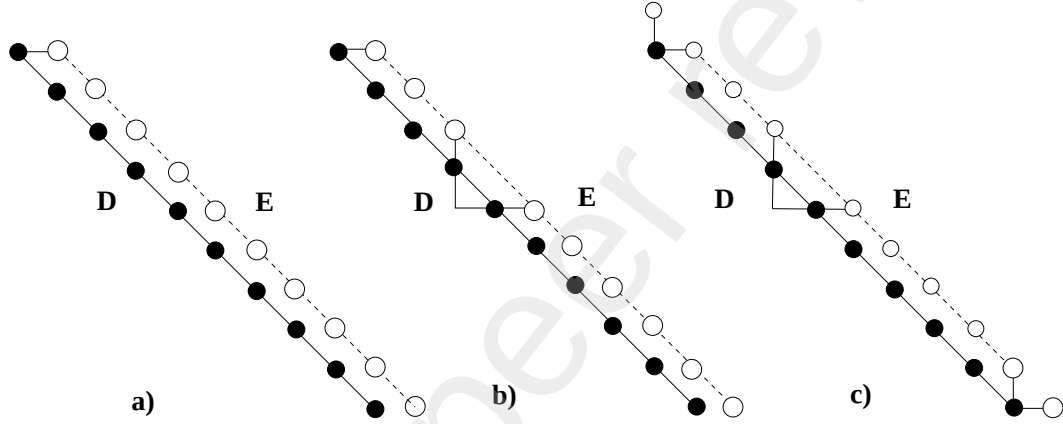


Figure 8: Moving guards from one diagonal to an adjacent diagonal.

where this kind of search, which we call a *simple* search, is optimal.

However, a simple search is not always optimal. Figure 6 shows a counter example. In this instance the simple sweep width is 14 in both orientations, i.e., $14 + 1$ guards are required. However, a better search sweeps through the trapezoid at the lower left first, using a positively orientated diagonal, leaving four guards to seal off the upper part of the grid. Once the trapezoid is cleared, the sealing guards can clean the upper part of the grid with a negatively orientated diagonal. This so called *composite search* uses only $10 + 1$ guards.

3.2 Relating Simple Sweepwidth to Search Number

We write sn for *search number* and ssw for *simple sweep width*. The next lemma establishes an upper bound on search number in terms of $ssw(G)$.

Lemma 3.1 *If G is a solid, convex grid then $sn(G) \leq ssw(G) + 1$.*

Proof

Without loss of generality let the sweep line be negatively orientated and proceed from left to right. We describe how to derive a search S for G from the sweep process, such that

$cost(S) = ssw^-(G) + 1$. The search is derived by noting that a sweep set is a separator so that, if each vertex in a sweep set is occupied by a guard, the set of guards seals off the clean from the dirty edges. Let D and E be adjacent diagonals. We show how to move guards from the vertices in D to the vertices in E , using no more than $|D_m^-| + 1$ guards.

Each of D and E may be continuous or discontinuous, as defined in Section 2.2. See Figure 7. Let D denote the diagonal being moved and E the destination diagonal. Then $|D| \leq |D_m^-|$ and $|E| \leq |D_m^-|$.

The discontinuities divide each diagonal into segments where each segment is a maximal continuous diagonal. Adjacent segments in a diagonal do not overlap but the elements of a single diagonal in D may be adjacent to elements in more than one segment in E . It is sufficient to consider D as just one segment and the vertices adjacent to D as E , whether they constitute one or more segments. See Figure 8.

If $|D| < |D_m^-|$ then, starting at say the uppermost vertex D and since no vertex in D has more than two neighbours in E , two extra guards are sufficient to move each guard from D to E , no more than $|D_m^-| + 1$ guards being used overall.

If $|D| = |D_m^-|$ then $|D| \geq |E|$. If $|D| = |E|$ then either one end vertex in D is unicritical, see Figure 8 a), or there exists a gap in E , Figure 8 c) and there exist unicritical vertices in the interior of D . Then all guards in D can be moved to E using just one extra guard by starting at a unicritical guard in D .

If $|D| > |E|$ then either there exists a gap in E , Figure 8 b), and hence unicritical vertices in the interior of D , or there exist unicritical vertices D . Hence again the move to E can be accomplished with just one extra guard.

Hence all vertices in any single segment can be moved to E using no more than one extra guard and so $|D_m^-| + 1$ guards suffice. By symmetry, the same argument holds for positively orientated sweeping and so $sn(G) \leq \min\{ssw^-(G), ssw^+(G)\} + 1$. $\square$

3.3 Composite Searching

For any diagonal $D \in \mathcal{D}^-$, consider a modified search which sweeps a positively orientated diagonal, E , through the grid G from left to right. The modification is that no guard is placed on any vertex strictly above the line defined by D and no guard placed on a vertex in D is removed. This process cleans all edges incident with any vertex in or strictly below the line defined by D and leaves $|D|$ guards on G .

See Figure 3.2 a) - d), where the shaded areas represent clean components and the circles represent guards. Figures a) - d) illustrate intermediate configurations before the completion of the sweep. Figure e) shows the completion of the sweep where all edges below D are clean. Let us call this search a *partial lower search* with respect to D . The partial lower search can sweep from left to right across D , denoted $PS_l^+(D)$, or from right to left, denoted $PS_l^-(D)$. We define the cost of partial search to be the maximum number of guards on the grid during the partial search. Then we define $cost(PS_l(D)) = \min\{cost(PS_l^-(D)), cost(PS_l^+(D))\}$. Likewise, we define a partial *upper* search to clean all edges above the line through D , denoted by $PS_u(D)$ and also upper and lower searches for any $D \in \mathcal{D}^+$.

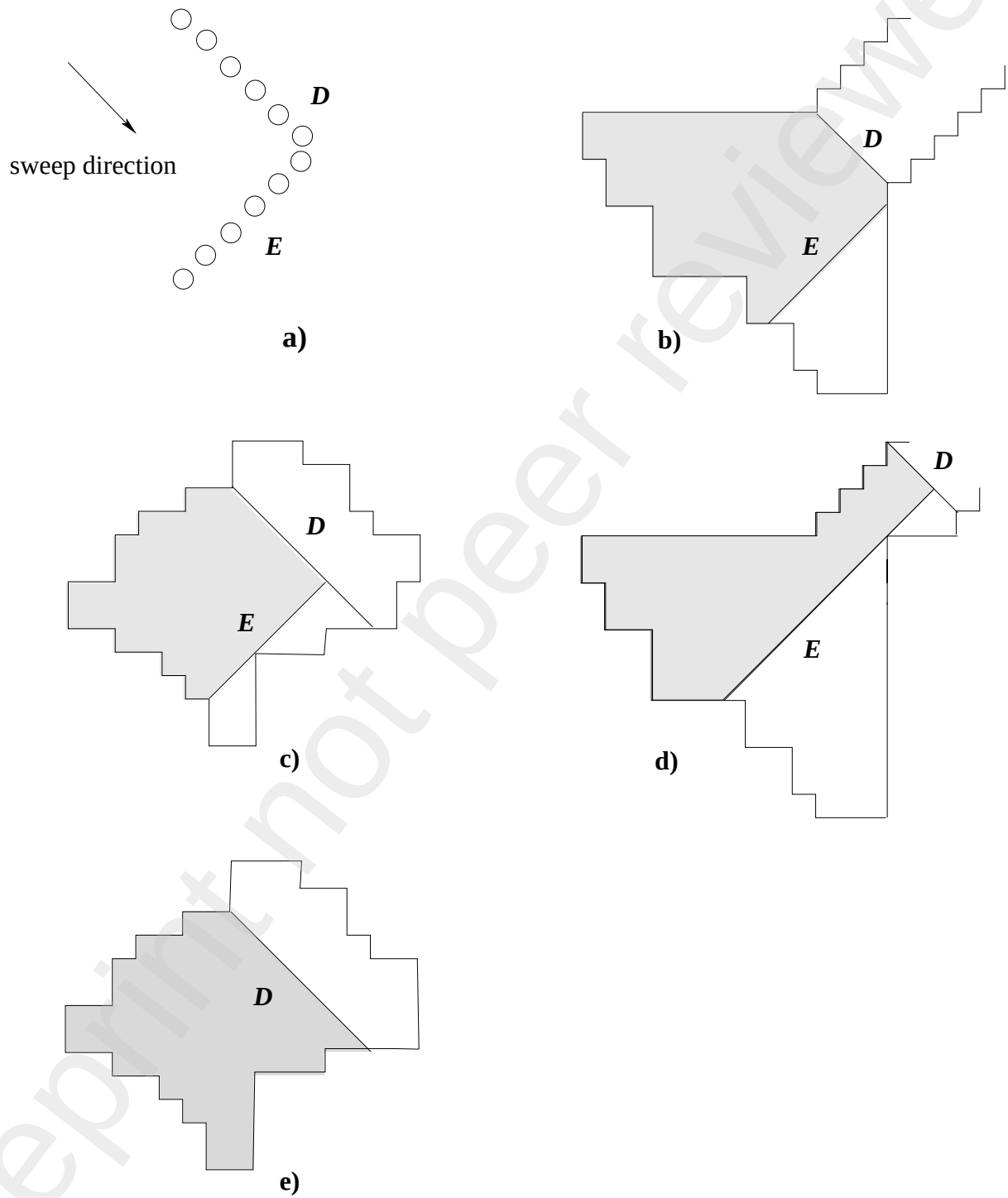


Figure 9: Composite searches

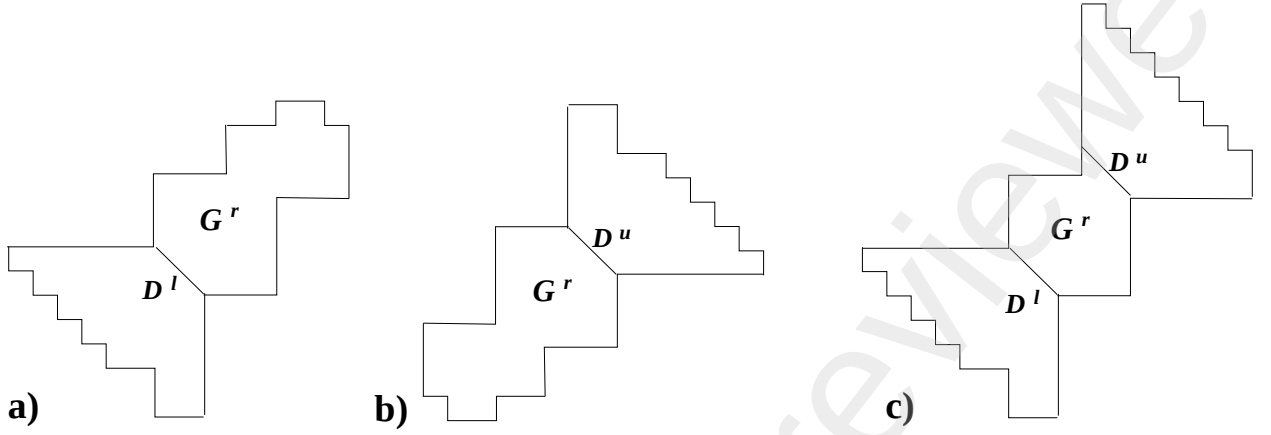


Figure 10: Composite searches at either and both ends.

Suppose there exist one or more $D^l \in \mathcal{D}^-$ having the property that $\text{cost}(PS_l(D^l)) < |D^l| + 1$ for all diagonals D' strictly below D^l . Of all such D^l , if any, that have this property, let D_m^l be the uppermost that minimises $\text{cost}(PS_l(D^l))$. Then it is possible that a lower partial search on D_m^l followed by a simple search of the remaining part of G , denoted G^r , above D_m^l can be done at a total cost less than $\text{ssw}^-(G) + 1$. See Figure 10 a). An example has already been shown in Figure 6. We call this a *composite search* and we say that the search is composite with respect to D_m^l .

There may also exist $D^u \in \mathcal{D}^-$ having the property that $\text{cost}(PS_u(D^u)) < |D^u| + 1$ for all diagonals D' strictly above D^u . Defining D_m^u similarly, it is then possible that an upper partial search on D_m^u followed by a cleaning of G^r below D_m^u , can be done at a total cost less than $\text{ssw}(G) + 1$. See Figure 10 b). It follows from the definitions of D_m^l and D_m^u that there can be no more than one example of each in any grid.

Composite searches may be possible at both ends of G as in Figure 10 c). Assuming a particular orientation, if just one end of G admits a composite search we call it a *single ended* grid. If G admits a composite search at both ends we call it a *double ended* grid. We note that, since D^l and D^u are defined to be, respectively, the upper most and lowmost diagonals, both of the same orientation, with the required properties, it is not possible for there to exist composite searches for both sweep orientations.

3.4 An Upper Bound on $\text{sn}(G)$

Without loss of generality, if G is single ended let the composite search be at the lower end. If the grid admits a composite search, let G^r denote the subgraph of G comprising the edges with neither end-point strictly below D_m^l and also, if it is double ended, those with neither end-point strictly above D_m^u . We then have the following lemma covering simple and composite searches. See Figure 10.

Lemma 3.2 Let $sn^-(G)$ denote the search number of a grid G as determined by diagonals in $\mathcal{D}^-$.

1. If G does not admit a composite search then $sn^-(G) \leq ssw^-(G) + 1$.
2. If G is single ended then $sn^-(G) \leq \max\{cost(PS_l(D_m^l)), ssw^-(G^r) + 1\}$.
3. If G is double ended then $sn^-(G) \leq \max\{cost(PS_l(D_m^l)), cost(PS_u(D_m^u)), ssw^-(G^r) + 1\}$.

Proof

1. If G does not admit a composite search then Lemma 3.1 suffices.
2. If G is single ended then that part of G below D^l can be cleaned using $cost(PS_l(D_m^l))$ guards while leaving $|D_m^l|$ guards on G . G^r can then be cleaned using $ssw^-(G^r) + 1$ guards, by Lemma 3.1.
3. If G is double ended, we note that G^r can be cleaned using $ssw^-(G^r) + 1$ guards while leaving $|D_m^u|$ guards on D_m^u . That part of G above D_m^u can be cleaned using $cost(PS_u(D_m^u))$ guards while leaving $|D_m^u|$ guards on G , starting from an empty graph. As mentioned in Section 1.2, any search can be reversed without increasing the cost. Hence that part of G above D_m^u can be cleaned, using $cost(PS_u(D_m^u))$ guards, by starting with all the vertices of D^u occupied by guards, running $PS_u^R(D_m^u)$ and ending with an empty grid. $\square$

By symmetry, analogous equations hold for $sn^+(G)$. Then $sn(G) \leq \min\{sn^-(G), sn^+(G)\}$. For convenience in the next section of the paper, the partial search part of the cost for a composite grid will be called the *composite search number* or *csn*.

4 Lower Bounds

From [9] and [4] we know that if G is a (w, w) -grid then $pw(G) \geq w$. In this section we extend that result by establishing a lower bound on the pathwidth of all solid, convex grids. It will be convenient to modify the meaning of a search *move* to be a guard placement immediately followed by the removal of any and all guards made redundant by that placement, where a *redundant* guard in one occupying a vertex all of whose incident edges are clean. Then, after the completion of any move in an optimal search, there are no more than $pw(G) = sn(G) - 1$ guards on G . We continue to assume that $|D_m^+| \leq |D_m^-|$ and we will use D_m to denote $\min D_m^+, D_m^-$.

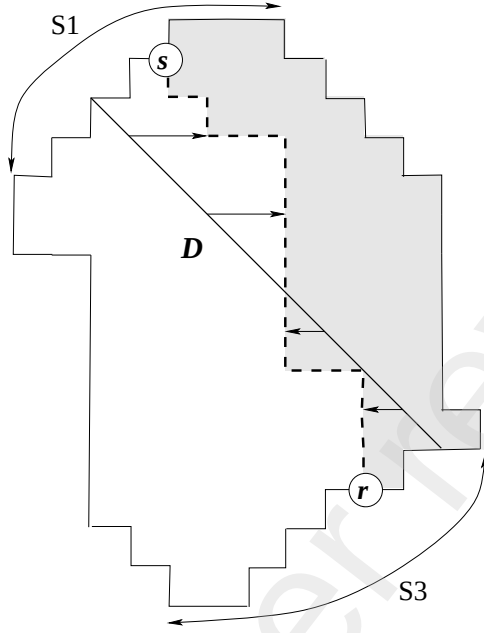


Figure 11: Paths intersecting D_m

4.1 Maximum Diagonals

Observation 4.1 *If there exists a clockwise, exposed path $P^{r,s}$ between opposing sides and that path includes a vertex in a diagonal D between those sides then $\text{critical}(P^{r,s}) \geq |D|$.*

Proof

See Figure 11, where the shaded faces are clean. Suppose that the maximum diagonal is negatively orientated. then, for all v in D_m^- :

- If v is below or in $P^{r,s}$ then the rightmost vertex in the horizontal line through v and in $P^{r,s}$ is critical.
- If v is above $P^{r,s}$ then the leftmost horizontal line through v and in $P^{r,s}$ is critical.

Hence for all v in D_m^- , there is a distinct critical vertex in $P^{r,s}$. $\square$

In a solid, convex grid G , let $\mathcal{D}^-$ and $\mathcal{D}^+$ denote, respectively, the sets of negatively and positively orientated diagonals in G . Let D_m^- and D_m^+ denote maximum diagonals in $\mathcal{D}^-$, $\mathcal{D}^+$ respectively. By symmetry and without loss of generality, we may assume that $|D_m^-| \geq |D_m^+|$. The *intersection* of diagonals was defined in Section 2.2.

Lemma 4.1 *There exists a maximum D in $\mathcal{D}^+$ and a C in $\mathcal{D}^-$ such that D and C intersect and $|C| \geq |D|$.*

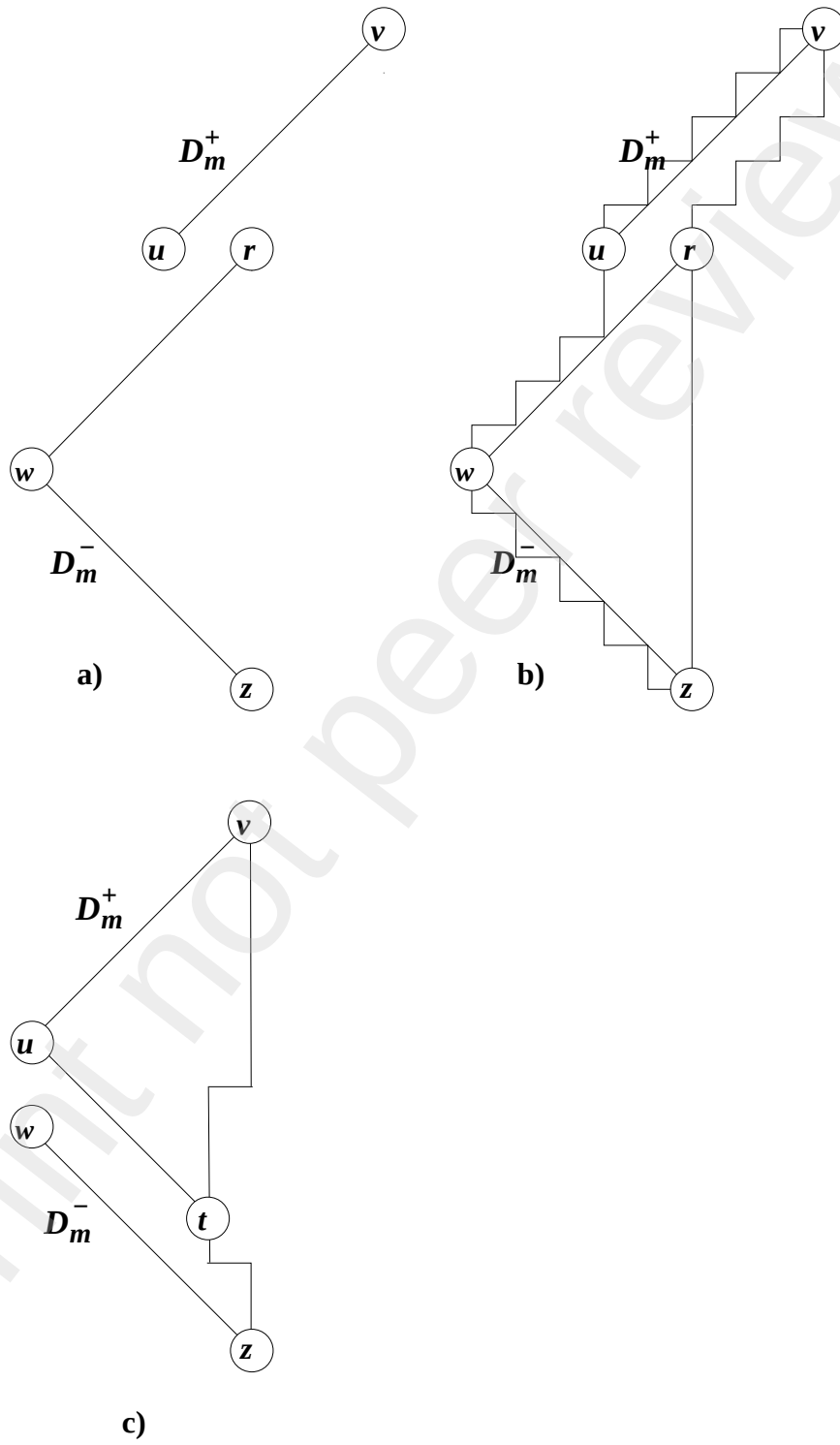


Figure 12: Diagonals intersecting D_m^+

Proof

Consider any maximum D in $\mathcal{D}^+$ and let us denote that particular diagonal by D_m^+ with end vertices u and v . If there exists a D in $\mathcal{D}^-$ that, with respect to D_m^+ , satisfies the requirements stated in the lemma, we are done. Else, for all D in $\mathcal{D}^-$ that intersect D_m^+ , $|D| < |D_m^+|$. See Figure 12.

By assumption, there exists a D_m^- , with endpoints w and z , in G such that $|D_m^-| \geq |D_m^+|$. If D_m^- does not intersect D_m^+ , we may assume, without loss of generality, that it lies below the lowermost diagonal that intersects D_m^+ . We consider two cases: $w_x < u_x$ and $w_x \geq u_x$.

Suppose $w_x < u_x$ as in Figure 12(a) and consider the diagonal in $\mathcal{D}^+$ that contains w , denoted by $D_{w,r}$. Since the path $P^{v,r,z}$ is convex, $r_x \geq z_x$ and hence $|D_m^-| \leq |D_{w,r}|$. But, by assumption, $|D_m^-| \geq |D_m^+|$, which is maximum. Hence $|D_{w,r}| \geq |D_m^+|$ and both are maximum. Then, since D_m^- is maximum in $\mathcal{D}^-$ and intersects $D_{w,r}$, they satisfy the lemma. A possible complete grid is illustrated in Figure 12(b).

If $w_x \geq u_x$, as in Figure 12(c), then $|D_{u,t}| < |D_m^+|$ and so $t_x < z_x$ and the peripheral path $P^{v,t,z}$ is not convex. By symmetry, the same arguments hold if D_m^- is above the uppermost, positively orientated diagonal that intersects D_m^+ . $\square$

4.2 Filling Corners

We define a *corner* in a simple grid to be a pair of adjacent, peripheral edges which form a 90° angle outside the grid, for example the peripheral edges in the face containing the dashed lines in Figure 13 a). By *filling a corner* in a grid we mean adding a vertex and two edges so as to remove a corner without increasing $|D_m^+|$. For example we can add a vertex v and edges to G to produce G' in Figures 13 a) and b).

Lemma 4.2 *Suppose G' is a grid produced by filling a corner in a simple grid G and the added vertex does not increase $|D_m^+|$. Then $sn(G) \leq |D_m^+|$ implies $sn(G') \leq |D_m^+|$.*

Proof

Let G be a simple grid with a corner, as in Figure 13 a), where a corner is filled by vertex v . Suppose $sn(G) \leq |D_m^+|$ so that there exists a search S of G where $cost(S) \leq |D_m^+|$. We use S to construct a search S' for G' , which is G plus the vertex v and the edges f and g that fill a corner without increasing $|D_m^+|$, such that $cost(S') = cost(S)$.

Let step t be the first step in S at the completion of which both edges e and h are clean. See Figure 13 a) and c). Since at least one of e , h is dirty at step $t-1$, both u and z are guarded or both z and w are guarded at step t .

Let G'_t denote the subgraph of G induced by the set of clean edges in G at step t plus the edges f and g . We define S' on G' as follows. We may assume that G'_t is defined by some clockwise, exposed path $P^{r,s}$ where r and s are peripheral vertices, Figures 13 d) and e). We consider the three possibilities: r and s are in the same side, in adjacent sides, or in opposing sides, as illustrated in 14 a), b) and c). If r and s are in opposing sides then, by Observation 4.1, $P^{r,s}$ does not intersect any D_m^+ between those sides else $critical(G'_t) \geq |D_m^+|$

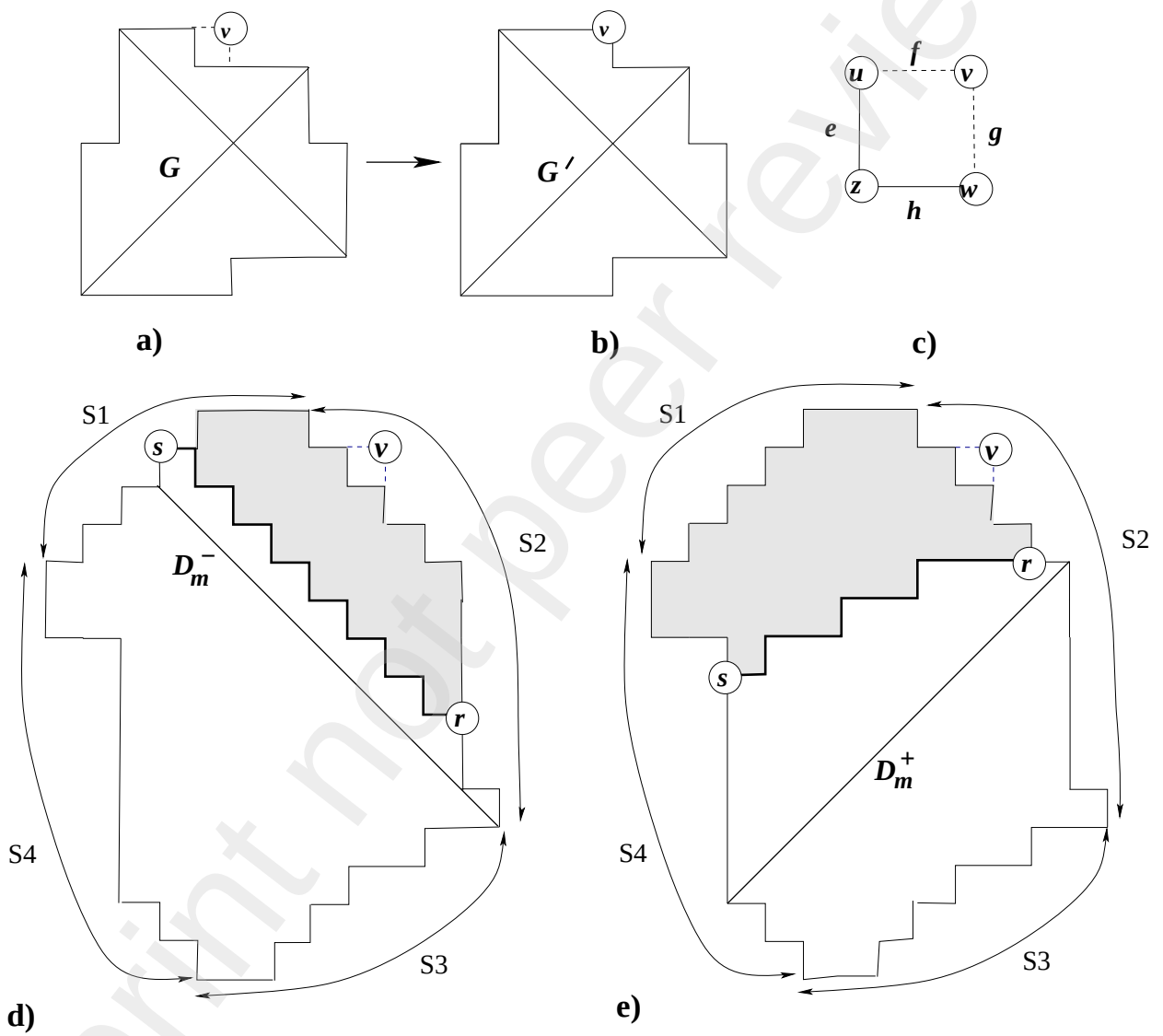


Figure 13: Filling Corners

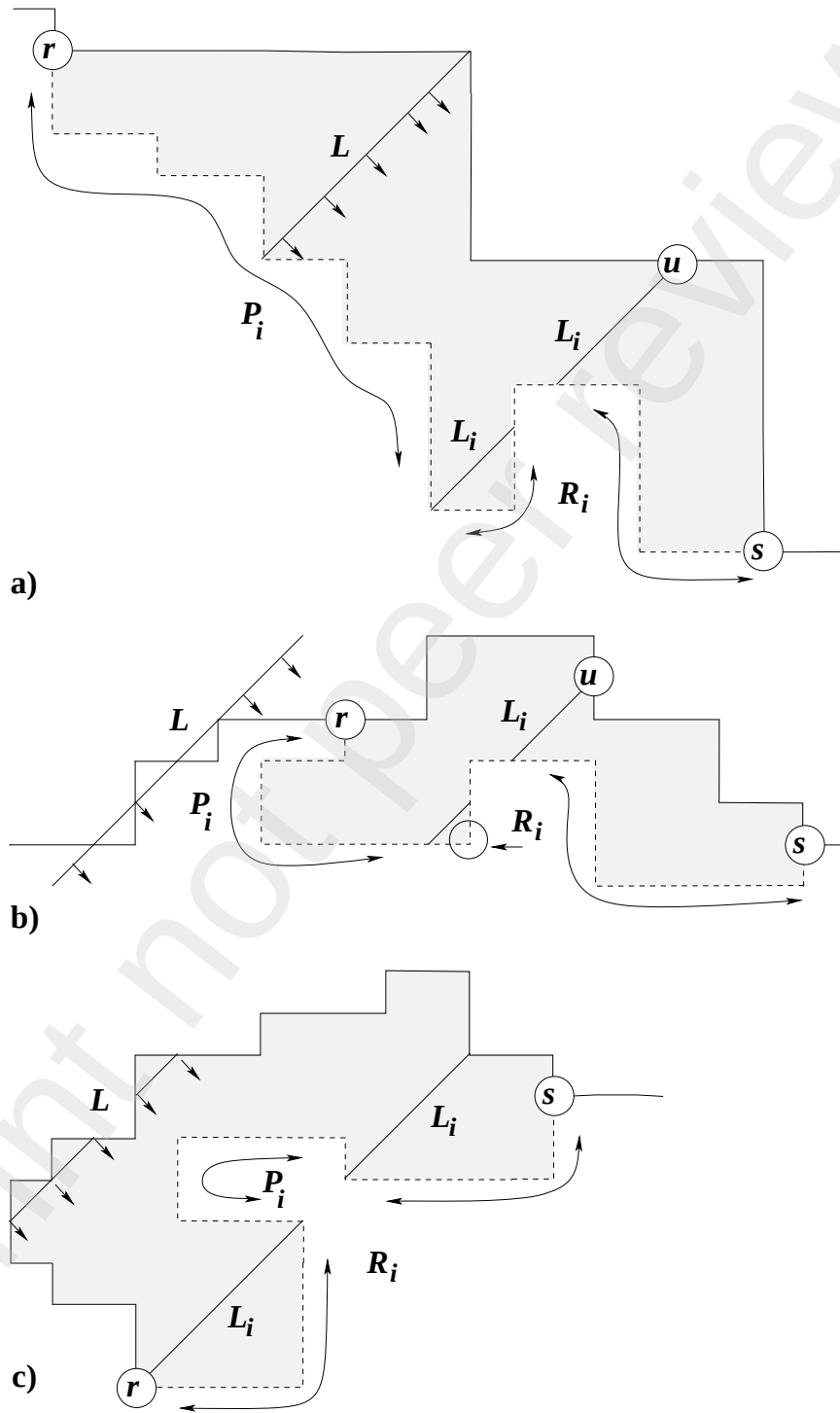


Figure 14: Cleaning a Segment

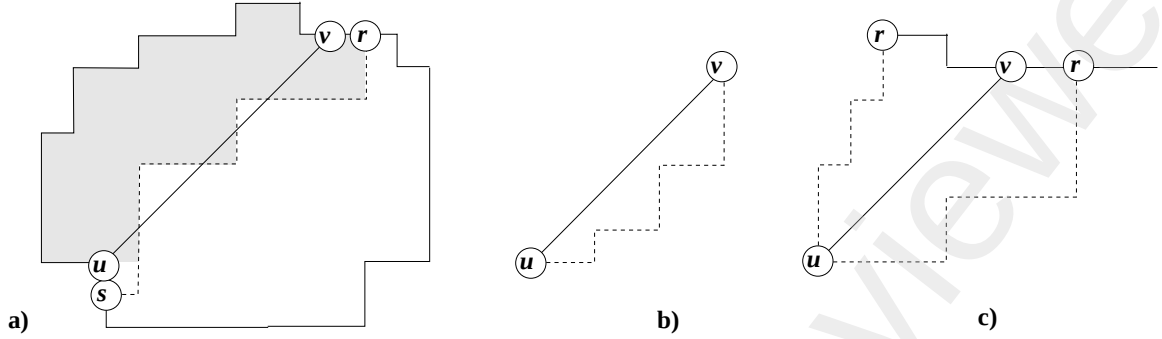


Figure 15: Sweep line and $P^{s,r}$

and $sn(G_t) \geq |D_m^+| + 1$ contradicting the initial assumption. Without loss of generality, we may assume the sides are $S2$ in the single case, $S1$ and $S2$ in the adjacent case and $S4$, $S1$ and $S2$ in the case of three sides.

By assumption, $cost(S) \leq |D_m^+|$ so $|critical(P^{r,s})| \leq |D_m| - 1$. Suppose G'_t is subjected to a downward sweep search of a positively orientated sweep line L . See Figure 14 a), b) and c). Let the i^{th} step in the search include all the guard movements from one diagonal to the adjacent diagonal as used in the proof of Lemma 3.1. Let L_i denote the set of vertices in the sweep line and in G'_t at the completion of the i^{th} step and let P_i denote the set of critical vertices in G'_t strictly above L_i , i.e., those guards placed in earlier steps on critical vertices. Let T_i denote the total number of guards on G'_t at the completion of the i^{th} step so that:

$$\forall i \ T_i = |P_i| + |L_i| \quad (1)$$

Let R_i denote the set of critical vertices in G'_t that are strictly below L_i , i.e., have not yet been occupied by a guard, at the completion of the i^{th} step. R_i divides L_i into one or more segments. For example, R_i in Figure 14 a) divides L_i into two segments. Consider Figure 15. Let v be the uppermost and u be the lowermost vertex in one of the L_i segments, denoted $D_{u,v}$, see Figure 15. If neither u nor v are in the periphery of G'_t , Figure 15 b), then $|D_{u,v}| \leq |P^{u,v}|$ because for all vertices w in $D_{u,v}$ there is a critical vertex in $P^{v,u}$ namely the lowermost vertex in the vertical line through w and in $P^{u,v}$. If v is in the periphery of G'_t , Figure 15 c), then, if $r_x < v_x$, the rightmost vertex in a horizontal line through w and in $P^{r,u}$ is critical. If $r_x > v_x$ then the lowermost vertex in the vertical line through w and in $P^{r,u}$ is critical. Hence:

$$\forall i \ |L_i| \leq |R_i| \quad (2)$$

From 1 and 2: $\forall i \ T_i \leq |P_i| + |R_i| = |critical(P^{r,s})|$. By assumption $|critical(P^{r,s})| \leq |D_m| - 1$ and so $\forall i \ T_i \leq |D_m| - 1$, i.e., G'_t can be cleaned using no more than $|D_m|$ guards.

Once the line sweep has cleaned G'_t the cleaning of G' can be completed by running S starting at step $t + 1$, at a total cost $\leq |D_m|$. $\square$

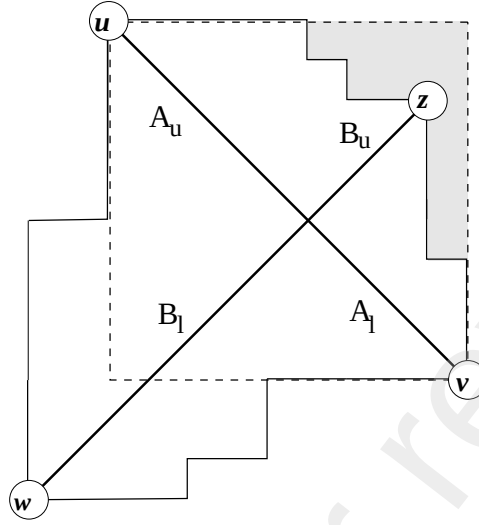


Figure 16: Creating a Square

Lemma 4.3 *If G is a simple grid then $sn(G) \geq |D_m| + 1$.*

Proof

If a (w, w) -grid is a subgraph of G then $sn(G) \geq w + 1$ otherwise consider Figure 16. Suppose G is simple and let A be a maximum diagonal in $\mathcal{D}^+$ and suppose the length of a maximum diagonal in $\mathcal{D}^- \geq |A|$. By Lemma 4.1 there exists a diagonal B in $\mathcal{D}^-$ such that B intersects $\mathcal{D}^+$ that intersects A and where $|B| \geq |A|$. The intersection defines upper and lower segments, A_u and A_l for A , and likewise for B . If all these segments are of equal length then G contains a square of side $|A|$, shown by the dashed lines, and so $sn(G) \geq |A| + 1 = |D_m^-|$. Otherwise we can orientate the diagram, as in Figure 16, with $z_y < u_y$, $z_x \leq v_x$ and $w_x \leq u_x$. With this orientation, every vertex in G that is in A or is strictly below A is in the square but some vertices that are strictly above A are not in the square.

For those that are not in G , by repeated application of Lemmas 4.2, we can fill the shaded section of the figure and expand G to a grid G' that is a supergraph of G and that does contain a (w, w) -grid as a subgraph, without increasing $|B|$. At each step in the expansion the property: if $sn(G) \leq D_m$ then the search number of the expanded graph is $\leq |D_m|$. Hence $sn(G) \leq |D_m|$, implies $sn(G_{|D_m|, |D_m|}) \leq |D_m|$ which is false. $\square$

Lemma 4.4 *If G is a composite grid then $sn(G) \geq csw(G)$.*

Proof

If G is composite then, Section 3.3, it contains a diagonal D such that the cost of a partial search on D is $< |D'|$ for all D' below D . See Figure 17. Let G' be the grid obtained by trimming all diagonals below D so that all length $\leq csw(G)$. Then, in G' , $|D_m| = csw(G)$.

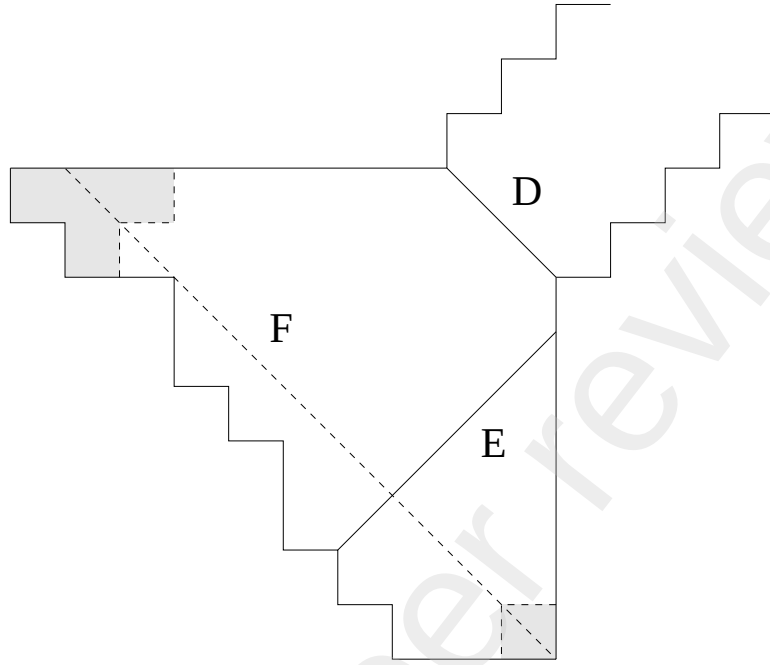


Figure 17: The composite case

Hence G contains a subgraph G' such that G' is simple and where $|D_m| = csw(G)$. The lemma then follows from Lemma 4.3. $\square$

From Lemmas 4.3, 4.4 and 3.2 we have:

Lemma 4.5 *If G is a solid, convex grid then:*
if G is simple grid then $sn(G) = |D_m| + 1$
if G is a composite grid then $sn(G) = csw(G)$.

$\square$

5 A Classification of Convex, Solid Grids

We partition the set of solid, convex grids into six classes. This will be useful because three of the classes will be shown to be simple. Since it is easy to check which class any particular grid belongs to, this implies that a simpler, faster algorithm can be used on grids in these three classes.

Consider traversing the perimeter of the grid in a clockwise fashion, noting the sequence of the left (L) and right (R) turns making up the traversal. There is a right turn at every one of the four corners and the number of right turns equals the number of left turns plus four. The sequence has the form:

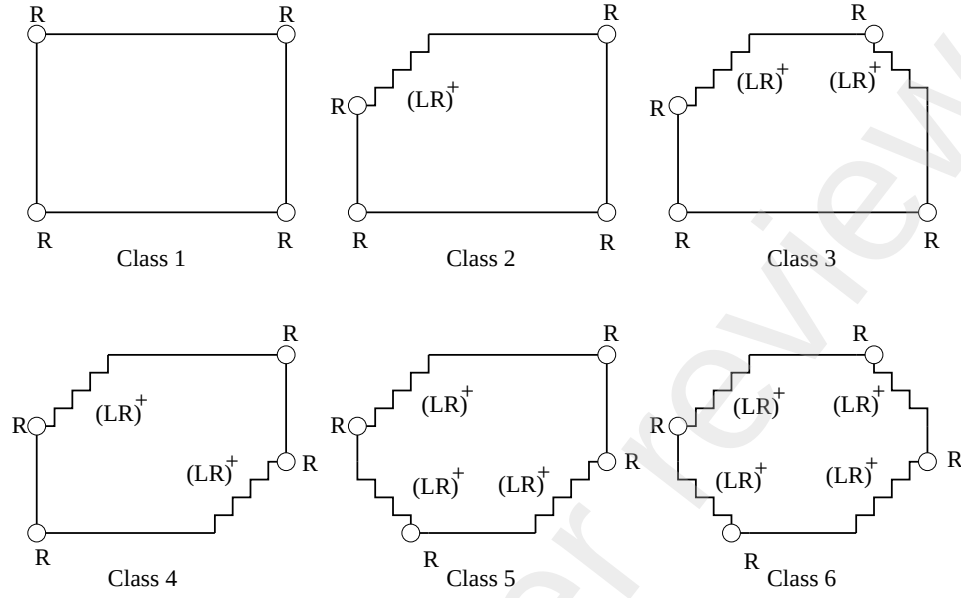


Figure 18: The six grid classes

$$R (LR)^* R (LR)^* R (LR)^* R (LR)^* ,$$

where $(LR)^*$ denotes a sequence of zero or more LR s. Since any of the $(LR)^*$ can be empty, there are six possibilities. See Figure 18.

Class 1 $R R R R$ – the perimeter is rectangular

Class 2 $R (LR)^+ R R R$ – one corner area is missing

Class 3 $R (LR)^+ R (LR)^+ R R$ – two adjacent corner areas are missing

Class 4 $R (LR)^+ R R (LR)^+ R$ – two opposite corner areas are missing

Class 5 $R (LR)^+ R (LR)^+ R (LR)^+ R$ – three corner areas are missing

Class 6 $R (LR)^+ R (LR)^+ R (LR)^+ R (LR)^+$ – all corner areas are missing

where $(LR)^+$ denotes a sequence of one or more LR s.

Lemma 5.1 *For all grids in Classes 1, 2 and 4, $pw(G) = ssw(G)$. For Classes 3, 5 and 6 there exist instances where $pw(G) < ssw(G)$.*

Proof

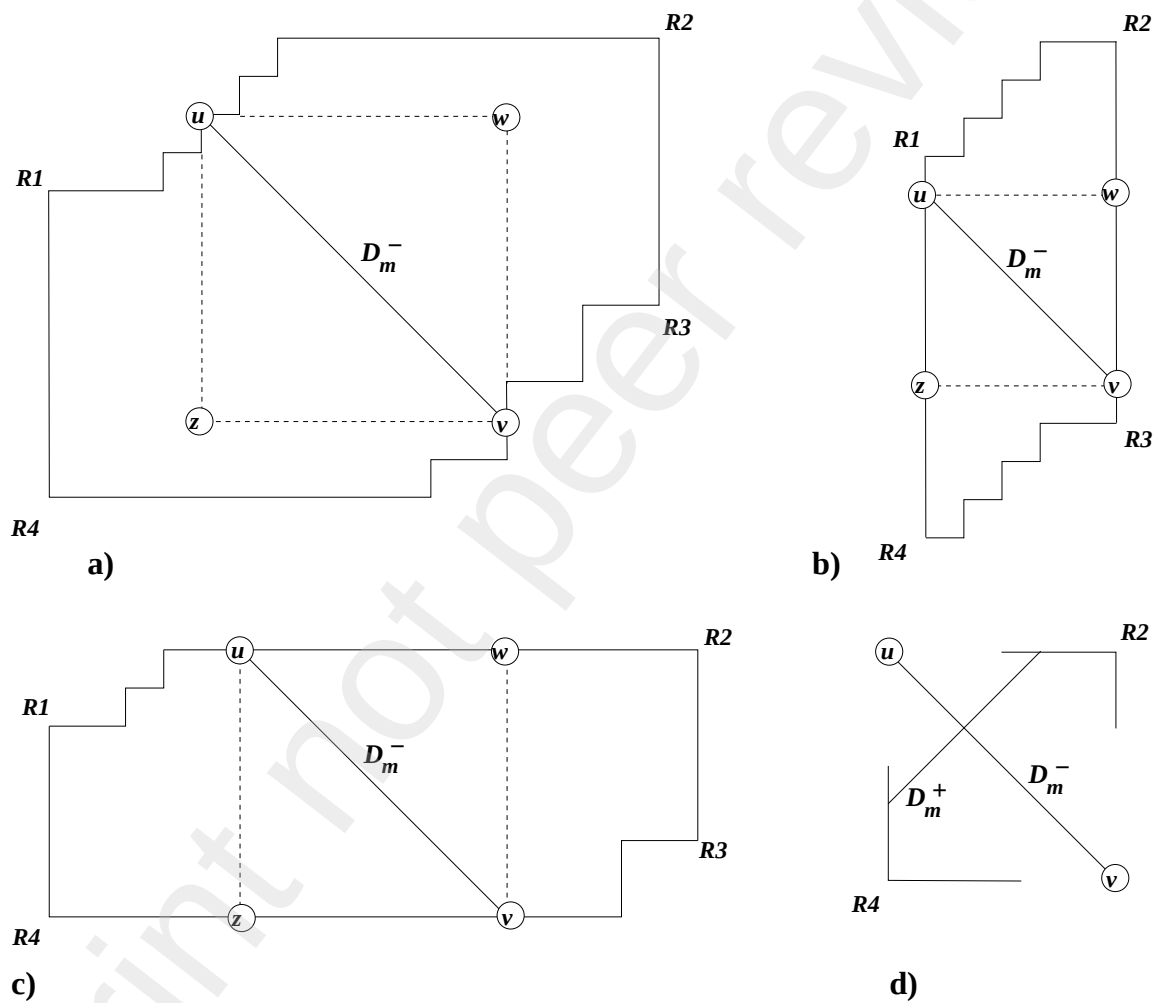


Figure 19: Class 4 grids are simple.

Class 1 Instances in this class are (h, w) -grids, as defined in Section 3.1. The pathwidth of an (h, w) -grid, as shown in Theorem 4.1 of [9], is $\min\{h, w\}$. The simple sweep width is also $\min\{h, w\}$.

Class 2 The proof for Class 4 is also valid for this class.

Class 3 Figure 20 shows a Class 3 grid for which the best simple search requires $(9 + 1)$ guards but for which there is composite strategy requiring only $(8+1)$ guards.

Class 4 Without loss of generality, let it be the upper left and lower right corner areas that are missing. Let D_m^- be a maximum diagonal in $\mathcal{D}^-$. See Figure 19 d). Let the upper end-point of D_m^- be u and let v be the lower end-point. It follows from the definition of this class that $R2$ is both the uppermost and rightmost vertex in G and $R4$ is both the leftmost and lowermost. Hence $u_y \leq R2_y$, $u_x \geq R4_x$, $v_y \geq R4_y$ and $v_x \leq R2_x$.

Consider the vertices w , where $w_x = v_x$, $w_y = u_y$ and z , where $z_x = u_x$ and $z_y = v_y$. See Figure 19 a), b) and c). Then $z_x < v_x$, $z_y < u_y$, $w_x > u_x$ and $w_y > v_y$.

Since $u_y \leq R2_y$ and $w_y = u_y$, $w_y \leq R2_y$. Since $v_x \leq R2_x$ and $w_x = v_x$, $w_x \leq R2_x$. Similarly, $z_y \geq R4_y$ and $z_x \geq R4_x$.

Hence w and z are in G and, since the grid is solid and convex, the square defined by the vertices u, w, v, x is a subgraph of the grid. It follows that, unless w is $R2$ and z is $R4$, in which case we have Class 1, $|D_m^-| > |D_m^+|$. Hence $pw(G) \geq |D_m^-|$, by Theorem 4.1 in [9] and so $pw(G) = ssw(G)$, by Lemma 3.1.

Classes 5 and 6 We have already shown instances of grids in classes 5 and 6 for which the optimal strategies are necessarily composite. See Figure 10. $\square$

6 The Time Complexity of an Algorithm

In this section, we put an upper bound on the time complexity of an algorithm based on the “sweep line” processes described in earlier sections.

As shown in Section 5, we can distinguish the different grid classes in time linear in the length of the perimeter of G , i.e., in $O(n)$ time where n is the number of vertices. We have shown, Lemma 5.1, that $ssw(G) = pw(G)$ for grids in Classes 1, 2 or 4. The “sweep line” process described in Section 3.2 can compute $ssw(G)$ and requires only the computation of D_m^- and D_m^+ for these classes. This requires only checking no more than every vertex in every diagonal using $O(n)$ time on a suitable representation of the grid.

For grids in Class 3, 5 and 6, we must consider the possibility that there is a composite sweep that is superior to the simple sweep. This can be done by, for all diagonals D in the simple sweep, computing $cost(PS(D))$ as defined in Section 3.4. Then it is only necessary to compare the current value of $cost(PS(D))$ to the largest value of $|D|$ and to the minimum value of $cost(PS(D))$ found so far, updating these values as appropriate. In this case each

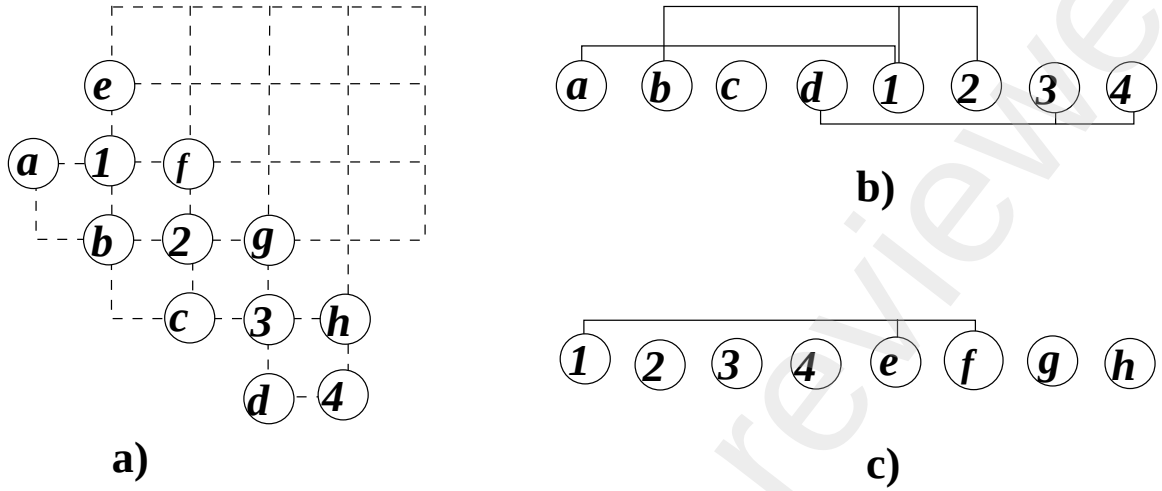


Figure 21: Building a layout from diagonals

diagonal orientation must be swept in both directions, because the value of $cost(PS(D))$ is dependent on both the direction and the orientation of the sweep. Since there are $O(n)$ diagonals and it requires time $O(n)$ to compute each corresponding $cost(PS(D))$, time $O(n^2)$ suffices.

7 Bandwidth

For $G = (V, E)$ such that $|V| = n$, a *linear layout* of G is a 1-1 function $L : V \rightarrow \{1, \dots, n\}$. The *bandwidth* of a graph is a linear layout property. The bandwidth of a graph G under a layout L , denoted $bw_L(G)$ is $\max_{(u,v) \in E} \{|L(u) - L(v)|\}$. The bandwidth of G , denoted $bw(G)$, is $\min_{all L} (bw_L(G))$. In general $bw(G) \geq sn(G)$ and the difference can be large. For example, if G is the star graph on n vertices then $sn(G) = 2$ and $bw(G) = \lfloor n/2 \rfloor$. In this section we observe that if G is a solid, convex grid and G does not admit a composite search, i.e., $sn(G) = \min\{ssw(G), ssw^+(G)\} + 1$, then $sn(G) \leq bw(G) \leq sn(G) + 1$.

We build a layout L from the sequence of diagonals defined by the sweep line during a simple search as defined in Section 3.1. Figure 21 a) shows three contiguous, maximal diagonals in a grid G . Suppose we layout each diagonal in the sequences defined by the sweep line with the constituent vertices in order from top to bottom. The resulting layout for the first two diagonals is shown in Figure 21 b) and for the second and third diagonals in Figure 21 c).

We note that no vertex in one diagonal is connected to more than two vertices in the succeeding diagonal. Consequently, if D is a maximum diagonal, no edge between adjacent diagonals can contribute more than $|D| + 1$ to the bandwidth. Hence $bw_L(G) \leq |D| + 1$. See Figure 21 b) and c).

In the special case where the first vertex in the layout for a particular diagonal has just one neighbour in the succeeding diagonal, no edge between the diagonals can contribute more than $|D|$ to the bandwidth. Hence, if it is possible to layout all maximal diagonals so that the vertex with one neighbour is the first for that diagonal, then $bw_L(G) \leq |D|$. This is not always possible as illustrated in Figure 21 c). It is possible if, for all maximal diagonals, the vertex with one neighbour is the uppermost in the diagonal or, for all maximal diagonals, the vertex with one neighbour is the lowermost in the diagonal.

Hence, since $|D| = sn(G) - 1$ and $pw(G) = sn(G) - 1$, we have the following observation:

Observation 7.1

1. *If G admits only simple searches then $pw(G) \leq bw(G) \leq pw(G) + 1$.*
2. *If G admits only simple searches and there exists an ordering of diagonals, such that for all maximum diagonals of the optimal orientation, the vertex with one neighbour in the succeeding diagonal is the uppermost for that diagonal or for all maximal diagonals, the vertex with one neighbour in the succeeding diagonal is the lowermost for that diagonal then $pw(G) = bw(G)$.*

In [14] the authors introduce a restricted form of pathwidth called *proper pathwidth*. The definition of proper pathwidth is identical to the definition of pathwidth with the addition of one condition. They go on to show, Theorem 3.2, that the bandwidth of a graph equals its proper pathwidth.

8 Conclusions

We have described a relatively simple, $O(n^2)$ time algorithm for computing the pathwidth of an interesting family of subgraphs of the (h, w) -grids. We suggest some ways these results might be extended.

It might be possible to develop some concept of a three dimensional equivalent of the solid, convex grids, so that some algorithm passing a plane through this grid would compute the pathwidth.

It would be interesting to discover whether or not the removal of the solidity or convexity constraints makes the problem NP-complete.

It may be possible to reduce the time complexity of the algorithm on those instances that do not require a composite search to time proportional to the length of the periphery of G because it may be possible to compute the size of the diagonals by working with their end points alone. Other improvements might possibly bring down the time complexity of the general case, at least for the average case.

We noted that the problem is NP-complete for planar graphs in general and note that embeddings of planar graphs into grids has been studied. See for example [24]. So it may be possible to create a better approximation algorithm for some more complex planar graphs

by embedding them into a solid, convex grid, whose pathwidth is then easy to compute.

Acknowledgement

The author wishes to thank Hongbing Fan for suggesting the grid classification scheme described in Section 5.

References

- [1] B Alspach. Searching and sweeping graphs: A brief survey. *Le Matematiche*, 59:5–37, 2004.
- [2] D Bienstock. Graph searching, path-width, tree-width and related problems (a survey). *DIMACS Series in Discrete Mathematics and Theoretical Computer Science*, 5:33–49, 1991.
- [3] H. Bodlaender and F. Fomin. Approximation of pathwidth of outerplanar graphs. *Journal of Algorithms*, 43:190–200, 2002.
- [4] Hans L. Bodlaender, Alexander Grigoriev, and Arie M. C. A. Koster. Treewidth lower bounds with brambles. *Algorithmica*, 51(1):81–98, 2008.
- [5] Hans L. Bodlaender and Ton Kloks. Efficient and constructive algorithms for the path-width and treewidth of graphs. *Journal of Algorithms*, 21(2):358–402, 1996.
- [6] L. Sunil Chandran and Telikepalli Kavitha. The treewidth and pathwidth of hypercubes. *Discrete Mathematics*, 306(3):359–365, 2006.
- [7] Josep Díaz, Jordi Petit, and Maria J. Serna. A survey of graph layout problems. *ACM Computing Surveys*, 34(3):313–356, 2002.
- [8] J Ellis and M. Markov. Computing the vertex separation of unicyclic graphs. *Information and Computation*, 192:123–161, 2004.
- [9] J. Ellis and R. Warren. Lower bounds on the pathwidth of some grid-like graphs. *Discrete Applied Mathematics*, 156(5):545–555, March 2008.
- [10] J. A. Ellis, I. H. Sudborough, and J. S. Turner. The vertex separation and search number of a graph. *Information and Computation*, 113(1):50–79, Aug 1994.
- [11] F.V. Fomin and D. M. Thilikos. An annotated bibliography on guaranteed graph searching. *Theoretical Computer Science*, 399:236–245, 2008.
- [12] Rajeev Govindan, Michael A. Langston, and Xudong Yan. Approximating the path-width of outerplanar graphs. *Information Processing Letters*, 68(1):17–23, 1998.
- [13] J.O’Rourke. *Computational Geometry in C*. Cambridge University Press, Cambridge, UK, 1998.
- [14] Kaplan and Shamir. Pathwidth, bandwidth, and completion problems to proper interval graphs with small cliques. *SIAM Journal on Computing*, 25(3):540–561, 1996.
- [15] Nancy G. Kinnersley. The vertex separation number of a graph equals its path-width. *Information Processing Letters*, 42(6):345–350, 1992.

- [16] L. M. Kirousis and C. H. Papadimitriou. Searching and pebbling. *Theoretical Computer Science*, 47(2):205–218, 1986.
- [17] A. LaPaugh. Recontamination does not help to search a graph. *Journal of the Association for Computing Machinery*, 40(2):224–245, 1993.
- [18] B. Monien and I. H. Sudborough. Min Cut is NP-complete for edge weighted trees. *Theoretical Computer Science*, 58(1-3):209–229, 1988.
- [19] Yota Otachi and Ryohei Suda. Bandwidth and pathwidth of three-dimensional grids. *Discrete Mathematics*, 311(10-11):881–887, 2011.
- [20] K. Skodinis. Computing optimal linear layouts of trees in linear time. *Proceedings of the 8th Annual European Symposium on Algorithms*, pages 403–414, 2000.
- [21] K. Skodinis. Construction of linear tree-layouts which are optimal with respect to vertex separation in linear time. *Journal of Algorithms*, 47, 2003.
- [22] Karol Suchan and Ioan Todinca. Pathwidth of circular-arc graphs. In Andreas Brandstädt, Dieter Kratsch, and Haiko Müller, editors, *Graph-Theoretic Concepts in Computer Science, 33rd International Workshop, 2007, Dornburg, Germany, June 21-23, 2007. Revised Papers*, volume 4769 of *Lecture Notes in Computer Science*, pages 258–269. Springer, 2007.
- [23] A. Symvonis and S. Tragoudas. Searching a pseudo 3-sided solid orthoconvex grid. *International Journal of Foundations of Computer Science*, 4(4):325–353, 1993.
- [24] R. Tamassia and I. G. Tollis. Planar grid embedding in linear time. *IEEE Trans. Circuits and Systems*, CAS-36(9):1230–1234, 1989.